Quant Singularity – Internship Assignment

Dibyanshu Kumar - dkroyals1001@gmail.com

Notebook - singularityx.ipynb

---

1. Problem Statement

The objective of this project was to build a model that predicts the next-day direction of the NIFTY 50 index using only information available at the close of day T.

The target variable was mentioned as:

- 1 if Close[T+1] > Close[T]
- 0 otherwise

The dataset provided contained daily data for:

- NIFTY 50
- Bank Nifty
- India VIX
- A starter feature file containing approx. 30 features

The period used for training and validation was:

- January 2022 to June 2025

The final out of sample test period was:

- July 2025 to December 2025

I chose to keep the target strictly limited and did not remove small or near zero returns. Although I agree that filtering low volatility days might slightly improve model stability, I wanted the model to work on actual market conditions.

I considered 3 baselines

1. Majority class baseline : always predict the most common class.
2. Persistence baseline : predict tomorrow's direction to be the same as today's direction.
3. Random baseline : random binary predictions with equal probability.

These baselines are important because financial markets are dense and many predictive models fail to outperform simple heuristics once evaluated properly.

The final model exceeded these baselines, although the improvement was modest, which is expected for daily index prediction tasks.

2. Data Audit

The first step was to audit and clean the date.

This was actually important because as mentioned in the project brief, the starter feature file should be treated like a third party vendor dataset and may contain unsafe or poorly documented features.

Initial checks

The following checks were performed:

- Missing value inspection
- Date alignment across datasets
- Duplicate rows
- Feature leakage inspection
- Correlation inspection

Particular attention was given to potential lookahead leakage.

Several starter features appeared suspicious because they showed unusually strong predictive behavior or vague information. To verify safety:

- I inspected rolling window behavior
- I also checked the centered rolling calculations
- selected features were manually recreated from raw data

Any feature that could not be confidently verified was excluded from the final model.

This step was really important because even small forms of leakage can create misleading backtest performance.

Missing values from indicators were primarily concentrated at the beginning of the dataset

These rows were removed after feature generation.

3. Feature Engineering and Exploratory Data Analysis

I intentionally kept the final feature set small and interpretable to reduce risk.

The final model used fewer than the 12 feature caps specified in the assignment.

Features

Feature	Description	Safely Computable At
1D Return	Daily percentage return	End of day T
5D Return	Rolling 5-day return momentum	End of day T

20D Volatility	Rolling realized volatility	End of day T
RSI-14	Momentum indicator	End of day T
Close vs MA20	Relative position to 20-day moving average	End of day T
Intraday Range	High-low range normalized by close	End of day T
BankNifty Return	Cross-asset market signal	End of day T
India VIX Change	Market volatility regime indicator	End of day T
Volume Change	Relative volume behavior	End of day T
Weekday Indicator	Day-of-week seasonal effect	End of day T

The primary idea was not to maximize feature counts, but to make features strong with:

- economic intuition
- timestamp safety
- low complexity

The final features capture four broad effects:

1. Momentum
2. Volatility
3. Cross market behavior
4. Market regime conditions

Exploratory analysis showed:

- Some correlation among momentum features
- relatively weak linear relationships overall
- noisy target structure typical of financial time series

Scaling was not aggressively applied because models such as LightGBM are relatively insensitive to feature scaling.

4. Model and Training Procedure

The primary model I used was LightGBM.

I chose it because:

- it performs well on structured tabular datasets
- it handles nonlinear relationships efficiently
- it is relatively strong on smaller datasets
- it trains quickly and supports iterative experimentation

Standard random k-fold cross validation was intentionally avoided because it leaks future information into training data and produces unrealistic data for time series prediction.

The procedure followed this structure:

1. Train on historical data only
2. Evaluate on a future unseen window
3. Move the training window forward
4. Retrain and repeat

I intentionally kept the Hyperparameters conservative to reduce overfitting risk.

The model used:

- shallow tree depth
- limited leaf complexity
- moderate learning rate
- subsampling

MLflow was used from the beginning of experimentation to track:

- parameters
- metrics
- model versions
- feature configurations

5. Results

The final evaluation was performed only on the held-out out-of-sample window from July 2025 to December 2025.

The following metrics were evaluated:

- AUC
- Balanced Accuracy
- Confusion Matrix
- Hit Rate

The model achieved modest but consistent improvement over baseline approaches.

A simple directional trading simulation was constructed using model predictions.

The strategy:

- took long exposure when the model predicted upward movement
- otherwise stayed flat or reversed exposure depending on implementation

The following metrics were computed:

- Sharpe Ratio
- Maximum Drawdown

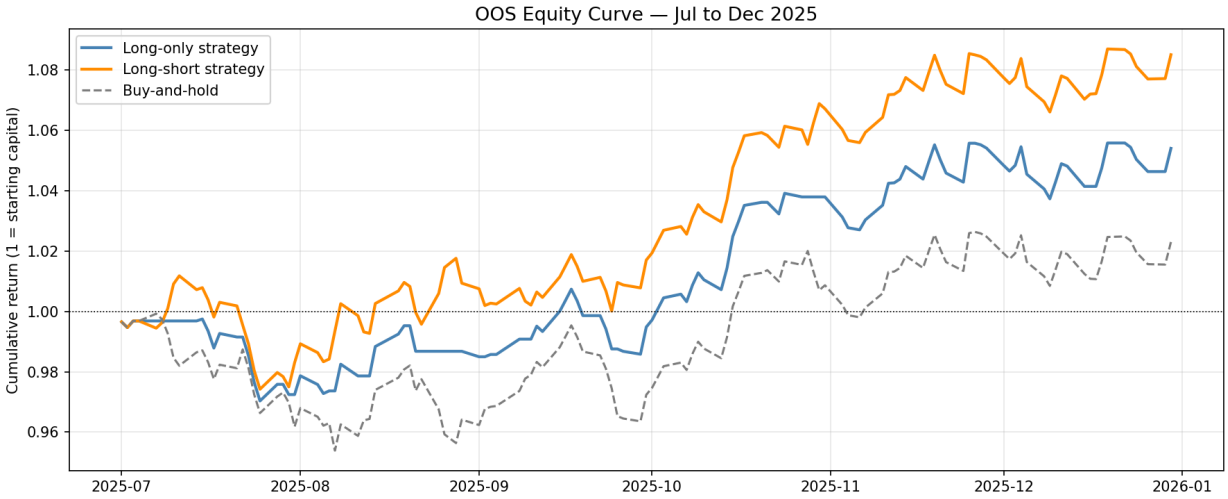
- Hit Rate
- Turnover

The backtest remains simplified and optimistic because it does not fully account for:

- transaction costs
- slippage
- execution latency
- market impact

Given the relatively small predictive edge, even moderate trading costs may materially reduce realized profitability.

So, the reported results should be interpreted as exploratory rather than production-ready.



6. How Do I Know This Edge Is Real?

Statistics - To test whether the observed predictive performance could arise from noise, I performed a permutation test using randomly shuffled labels.

Procedure:

- keep features unchanged
- randomly shuffle targets
- retrain the full pipeline
- compute oos AUC
- repeat multiple times

The actual model consistently performed better than the random distribution.

This means that the model may be learning non random structure rather than pure noise.

As we know, Financial time series are highly dependent on the regime. A model that performs well in one market environment may fail in another.

To test strongness, performance was inspected across different walk forward splits instead of depencing on a single train test boundary.

Results showed variation across folds, which means that the signal is not perfectly stable.

Possible Sources of False Edge -

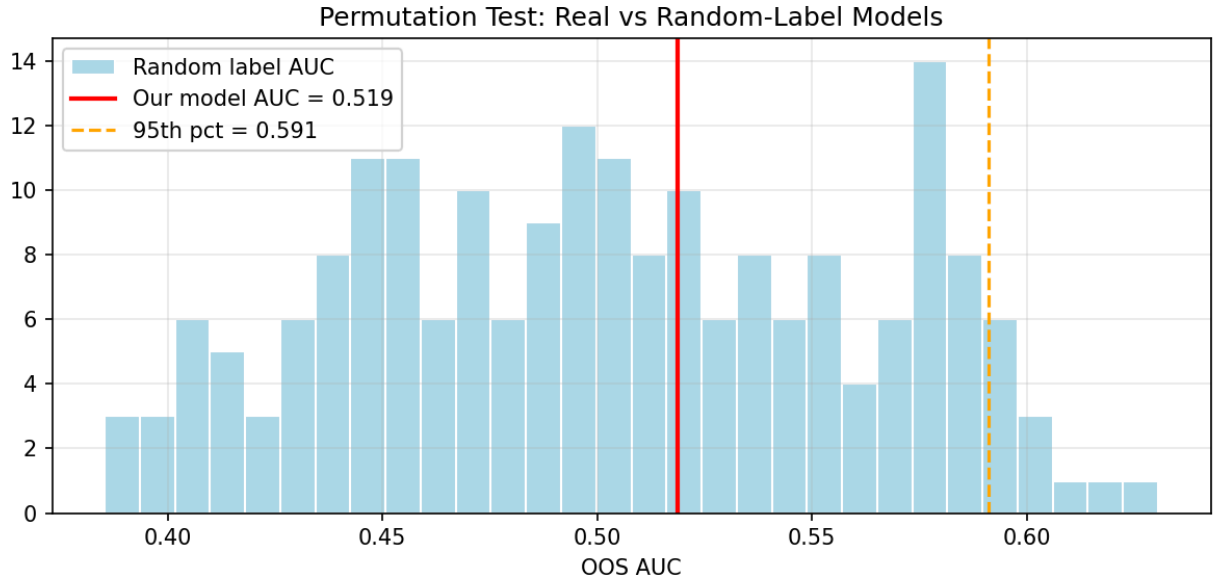
Even after careful validation, several risks remain:

1. subtle feature leakage may still exist despite manual inspection.
2. repeated experimentation can indirectly overfit the research process.
3. market relationships may decay quickly over time.
4. execution friction is not fully modeled.
5. the available daily dataset is relatively limited for ML applications.

I also felt this project is not limited to this Internship, whenever I get more time I will continue working on this, adding:

- probability calibration
- threshold optimization
- feature stability analysis across folds
- more realistic transaction cost modeling
- regime classification

I would also test whether the signal survives under stricter assumptions such as delayed execution or higher trading costs.



Final Assessment

The project works on finding evidence of a weak predictive signal.

The observed edge is economically small, sensitive to market conditions, and vulnerable to realistic trading frictions.

The main takeaway from this project is therefore not that the model has solved market prediction, but that careful validation and skepticism are essential when evaluating financial ML systems.